hi mujhe wazuh dashborad banan hai alert
traget hai ki
alerts trking system ilke bana hai
1 mujhe yah sab level 10 ka alert dekhana haii
2 kitan alert open kiy
3 itan mai note
4 tolal kitan alert aay ga hai

? keshe ho ga methos kitan hai sab kucha expaile and issues baatao

Namaste! Wazuh dashboard mein aapke requirements ke hisaab se alert tracking system banane ke liye main aapko step-by-step guide deta hoon.

Aapki Requirements:

1. **Level 10 ke alerts dikhana**
2. **Open alerts ki count**
3. **Alerts mein notes**
4. **Total alerts ka count**

Solutions & Methods:

Method 1: Using Wazuh Discover (Quick Setup)

Step 1: Level 10 Alerts Filter

Wazuh Discover mein jayein aur ye filter lagayein:

```
json

{
  "query": {
    "match": {
      "rule.level": "10"
    }
  }
}
```

KQL Query:

```
text

rule.level:10
```

Step 2: Dashboard Visualizations

Pie Chart/Donut Chart banaye:

1. **Total Level 10 Alerts:**
   - Visualization type: `Metric`
   - Aggregation: `Count`
   - Filter: `rule.level:10`
2. **Open vs Closed Alerts:**
   - Visualization type: `Pie`
   - Buckets: `Split slices`
   - Aggregation: `Terms`
   - Field: `rule.level`
   - Custom filter: `rule.level:10`

Alert Status Tracking ke liye:

```
yaml

# Wazuh manager config (/var/ossec/etc/ossec.conf)
<global>
  <jsonout_output>yes</jsonout_output>
  <alerts_log>yes</alerts_log>
  <logall>no</logall>
</global>
```

Method 2: Custom Script Integration

Python Script for Alert Management:

```
python

#!/usr/bin/env python3
import json
import requests
```

```python
from datetime import datetime, timedelta
import os

# Configuration
WAZUH_API = "https://YOUR_WAZUH_IP:55000"
USER = "wazuh-wui"
PASSWORD = "YOUR_PASSWORD"

class AlertTracker:
    def __init__(self):
        self.base_url = WAZUH_API
        self.token = self.get_token()
        self.notes_file = "/var/ossec/logs/alert_notes.json"

    def get_token(self):
        """Wazuh API se token lein"""
        url = f"{self.base_url}/security/user/authenticate"
        response = requests.post(url, auth=(USER, PASSWORD), verify=False)
        return response.json()['data']['token']

    def get_level_10_alerts(self, hours=24):
        """Level 10 alerts get karein"""
        headers = {'Authorization': f'Bearer {self.token}'}

        # Last 24 hours ke alerts
        end_time = datetime.now()
        start_time = end_time - timedelta(hours=hours)

        payload = {
            "filters": [
                {"rule.level": "10"},
                {"timestamp": f"{start_time.isoformat()}Z"}
            ]
        }

        url = f"{self.base_url}/alerts"
        response = requests.get(url, headers=headers, params=payload, verify=False)
        return response.json()

    def get_alert_stats(self):
        """Alert statistics nikalein"""
        alerts = self.get_level_10_alerts()

        stats = {
            "total_alerts": len(alerts.get('data', {}).get('affected_items', [])),
            "open_alerts": 0,
            "closed_alerts": 0,
```

```
            "by_rule": {}
        }

        for alert in alerts.get('data', {}).get('affected_items', []):
            rule_id = alert.get('rule', {}).get('id')

            # Status check (agar aapke paas custom field hai)
            if not stats['by_rule'].get(rule_id):
                stats['by_rule'][rule_id] = 0
            stats['by_rule'][rule_id] += 1

        return stats

    def add_note_to_alert(self, alert_id, note):
        """Alert mein note add karein"""
        notes = self.load_notes()
        notes[alert_id] = {
            "note": note,
            "timestamp": datetime.now().isoformat(),
            "status": "open"
        }
        self.save_notes(notes)

        # API ke through update karein
        headers = {'Authorization': f'Bearer {self.token}'}
        url = f"{self.base_url}/alerts/{alert_id}"

        payload = {
            "status": "noted",
            "comment": note
        }

        response = requests.put(url, headers=headers, json=payload, verify=False)
        return response.json()

    def load_notes(self):
        """Saved notes load karein"""
        if os.path.exists(self.notes_file):
            with open(self.notes_file, 'r') as f:
                return json.load(f)
        return {}

    def save_notes(self, notes):
        """Notes save karein"""
        with open(self.notes_file, 'w') as f:
            json.dump(notes, f, indent=2)
```

```
    def get_alert_summary(self):
        """Complete alert summary"""
        stats = self.get_alert_stats()
        notes = self.load_notes()

        open_count = len([n for n in notes.values() if n.get('status') == 'open'])

        return {
            "total_level_10_alerts": stats['total_alerts'],
            "open_alerts_with_notes": open_count,
            "alert_distribution": stats['by_rule'],
            "recent_notes": list(notes.items())[-5:]  # Last 5 notes
        }

# Usage
tracker = AlertTracker()

# Statistics lein
summary = tracker.get_alert_summary()
print(f"Total Level 10 Alerts: {summary['total_level_10_alerts']}")
print(f"Open Alerts: {summary['open_alerts_with_notes']}")

# Note add karein
tracker.add_note_to_alert("alert_123", "Critical alert - needs immediate attention")
```

Method 3: Elasticsearch + Wazuh Integration

Elasticsearch Pattern:

```
json

// Wazuh Index Pattern: wazuh-alerts-*
GET wazuh-alerts-*/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "rule.level": 10 }}
      ]
    }
  },
  "aggs": {
    "alerts_by_status": {
      "terms": { "field": "status" }
```

```
    },
    "total_level_10": {
      "value_count": { "field": "rule.level" }
    }
  }
}
```

Dashboard Panels Creation:

Panel 1: Alert Overview (Metric)

```
json

{
  "title": "Total Level 10 Alerts",
  "type": "visualization",
  "params": {
    "expression": ".es(index=wazuh-alerts-*, q='rule.level:10').count()"
  }
}
```

Panel 2: Alert Status Table

```
json

{
  "title": "Alert Tracking Table",
  "type": "table",
  "columns": [
    {"field": "rule.id", "title": "Rule ID"},
    {"field": "rule.description", "title": "Description"},
    {"field": "agent.name", "title": "Agent"},
    {"field": "timestamp", "title": "Time"},
    {"field": "status", "title": "Status"},
    {"field": "comment", "title": "Notes"}
  ]
}
```

Method 4: Complete Dashboard Setup (Wazuh UI)

Step 1: Custom Index Pattern

Wazuh Management → Index Patterns → Create:

- Name: `wazuh-alerts-custom`
- Time field: `timestamp`

Step 2: Saved Searches

```
kql

# Level 10 Alerts
rule.level:10

# Open Alerts (agar status field hai)
rule.level:10 AND status:open

# Alerts with Notes
rule.level:10 AND comment:*
```

Step 3: Create Visualizations

1. Metric Chart (Total Alerts):

```
text

Aggregation: Count
Custom Label: Total Level 10 Alerts
Color: Red
```

2. Data Table (Alert Details):

```
text

Columns:
- rule.id
- rule.description
- agent.name
- timestamp
- location
```

3. Tag Cloud (Alert Categories):

```
text

Tags Field: rule.groups
Size: Count
```

Step 4: Dashboard Layout

text

```
+-----------------------------------------+
| [TOTAL ALERTS METRIC] [OPEN ALERTS] [TODAY] |
+-----------------------------------------+
|          ALERTS BY AGENT (BAR CHART)      |
+-----------------------------------------+
|     ALERT TABLE WITH NOTES COLUMN         |
| Rule ID | Agent | Time | Note | Status    |
+-----------------------------------------+
|      ALERT TREND (LINE CHART)             |
+-----------------------------------------+
```

Method 5: Real-time Monitoring Script

python

```python
#!/usr/bin/env python3
# Alert Monitoring Daemon
import time
import sqlite3
from wazuh_api import WazuhAPI

class AlertMonitor:
    def __init__(self):
        self.api = WazuhAPI()
        self.db = sqlite3.connect('/var/lib/alert_tracker.db')
        self.init_db()

    def init_db(self):
        """Database initialize karein"""
        self.db.execute('''
            CREATE TABLE IF NOT EXISTS alerts (
                id TEXT PRIMARY KEY,
                rule_id TEXT,
                level INTEGER,
                agent TEXT,
                timestamp DATETIME,
                status TEXT DEFAULT 'open',
                note TEXT,
                assigned_to TEXT
            )
```

```
        ''')

    def monitor_alerts(self):
        """Real-time alert monitoring"""
        last_check = time.time() - 300  # 5 minutes

        while True:
            alerts = self.api.get_alerts_since(last_check, level=10)

            for alert in alerts:
                self.store_alert(alert)
                self.notify_if_critical(alert)

            last_check = time.time()
            time.sleep(60)  # Every minute check

    def store_alert(self, alert):
        """Alert database mein store karein"""
        self.db.execute('''
            INSERT OR IGNORE INTO alerts (id, rule_id, level, agent, timestamp)
            VALUES (?, ?, ?, ?, ?)
        ''', (
            alert['id'],
            alert['rule']['id'],
            alert['rule']['level'],
            alert.get('agent', {}).get('name', 'unknown'),
            alert['timestamp']
        ))
        self.db.commit()

    def get_stats(self):
        """Current statistics"""
        cursor = self.db.cursor()

        total = cursor.execute(
            "SELECT COUNT(*) FROM alerts WHERE level=10"
        ).fetchone()[0]

        open_alerts = cursor.execute(
            "SELECT COUNT(*) FROM alerts WHERE level=10 AND status='open'"
        ).fetchone()[0]

        with_notes = cursor.execute(
            "SELECT COUNT(*) FROM alerts WHERE note IS NOT NULL"
        ).fetchone()[0]

        return {
```

```
            "total": total,
            "open": open_alerts,
            "with_notes": with_notes
        }
```

Important Issues & Solutions:

Issue 1: Alert Volume High

Problem: Agar bahut zyada level 10 alerts aa rahe hain

Solution:

```xml
<!-- ossec.conf mein alert threshold set karein -->
<alerts>
  <log_alert_level>10</log_alert_level>
  <email_alert_level>10</email_alert_level>
  <!-- Alert deduplication -->
  <alert_deduplication>
    <time>3600</time> <!-- 1 hour -->
    <rule_id>100001,100002</rule_id>
  </alert_deduplication>
</alerts>
```

Issue 2: Performance

Problem: Dashboard slow ho raha hai

Solution: Index lifecycle management

```json
{
  "policy": {
    "phases": {
      "hot": {
        "min_age": "0ms",
        "actions": {"rollover": {"max_size": "50gb"}}
      },
      "delete": {
        "min_age": "30d",
```

```
        "actions": {"delete": {}}
      }
    }
  }
}
```

Issue 3: Notes Store Limitation

Problem: Wazuh native notes support limited hai

Solution: Elasticsearch enrichment

```
json

{
  "enrich": {
    "policy_name": "alert_notes_policy",
    "match_field": "alert_id",
    "enrich_fields": ["note", "analyst", "status"]
  }
}
```

Quick Setup Commands:

```
bash

# 1. Wazuh API token generate karein
TOKEN=$(curl -u wazuh-wui:password -k -X POST https://localhost:55000/security/user/authe
nticate | jq -r '.data.token')

# 2. Level 10 alerts fetch karein
curl -k -H "Authorization: Bearer $TOKEN" \
  "https://localhost:55000/alerts?filters=rule.level=10&limit=100" | jq '.'

# 3. Alert notes API call
curl -k -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"comment":"Critical investigation needed"}' \
  "https://localhost:55000/alerts/ALERT_ID/notes"
```

Best Dashboard Layout Suggestion:

```yaml
Dashboard Name: "Security Alert Tracker - Level 10"

Panels:
  Row 1:
    - Total Level 10 Alerts (24h) [Metric - Red]
    - Open Alerts [Metric - Orange]
    - Alerts with Notes [Metric - Blue]
    - Response Time [Metric - Green]

  Row 2:
    - Alert Timeline (Line Chart)
    - Alerts by Agent (Bar Chart)

  Row 3:
    - Alert Details Table [with Notes column]
    - Top Rules Pie Chart

  Row 4:
    - Recent Alerts Feed
    - Alert Categories Heatmap
```

Kya aapko specific part mein aur detail chahiye? Jaise:

- Database setup?
- API integration?
- Specific visualization?
- Custom script modifications?